Case Statement

Overview:

The SQL CASE Expression

The `CASE` expression goes through conditions and returns a value when the first condition is met (like an if-then-else statement). So, once a condition is true, it will stop reading and return the result. If no conditions are true, it returns the value in the `ELSE` clause.

If there is no `ELSE` part and no conditions are true, it returns NULL.

🔎 Explanation

- **CASE** lets you add conditional logic inside SQL queries.
- Each `WHEN ... THEN ...` is like an **if-else** branch.
- The `ELSE` handles all other cases.
- The result is returned as a new column (`income_category` here).

CASE Syntax

```
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    WHEN conditionN THEN resultN
    ELSE result
END;
```

Demo Database

Below is a selection from the "OrderDetails" table in the Northwind sample database:

OrderDetailID	OrderID	ProductID	Quantity
1	10248	11	12
2	10248	42	10
3	10248	72	5
4	10249	14	9
5	10249	51	40

Example 1: Simple CASE Expression

In this example, we use CASE statement

Query:

```
SELECT CustomerName, Age,
CASE
    WHEN Country = "India" THEN 'Indian'
    ELSE 'Foreign'
END AS Nationality
FROM Customer;
```

Output:

CustomerName	Age	Nationality
Shubham	23	Indian
Aman	21	Foreign
Naveen	24	Foreign
Aditya	21	Foreign
Nishant. Salchichas S.A.	22	Foreign

EXAMPLES

```sql
SELECT
    age,
    country,
    CASE
        WHEN age < 18 THEN 'Minor'
        WHEN age >= 18 AND country = 'USA' THEN 'Adult (USA)'
        ELSE 'Adult (Other)'
    END AS category
FROM persons;
```

INPUT:(Presons)

age	country
15	USA
25	USA
21	India

OUTPUT

age	country	category
15	USA	Minor
25	USA	Adult (USA)
21	India	Adult (Other)

⚡ Key Points to Remember

- **CASE is like IF-ELSE** inside SQL queries.
- You can use it in `SELECT`, `ORDER BY`, `GROUP BY`, and even `UPDATE`.
- If no condition matches and no `ELSE` is given, the result is **NULL**.

✅ Conclusion on CASE Statements

The **CASE statement** in SQL is a powerful and flexible tool used to add conditional logic directly inside queries. It allows you to:

- Categorize or transform data based on conditions
- Handle NULLs and exceptions
- Create calculated fields
- Perform conditional aggregations
- Build custom sorting rules
- Even influence JOIN, GROUP BY, HAVING, and ORDER BY logic

Overall, **CASE enhances SQL queries by enabling decision-making within the database**, reducing the need for application-side logic.

It is essential for advanced reporting, data validation, analytics, and real-world business rules.